

ROAD TO COLOSSEUM · SOLANA DEVNET · LIVE

Los agentes ya mueven plata. Nadie responde por ellos.

dnia — la capa de accountability para pagos de agentes IA: identidad verificada + mandato + recibo, exigidos **dentro** de la transacción que mueve la plata.

programa live en devnet

21/21 tests

reverts reales en explorer

EL PROBLEMA

Cuando un agente paga, nadie puede responder 3 preguntas

¿Quién está atrás?

identidad

Nada en una request distingue agente de humano ni dice quién lo opera. (IETF draft AIP lo formaliza.)

¿Qué autorizó?

mandato

El agente usa la wallet del dueño sin frontera de permisos. No hay límite, ni lista, ni revocación exigible.

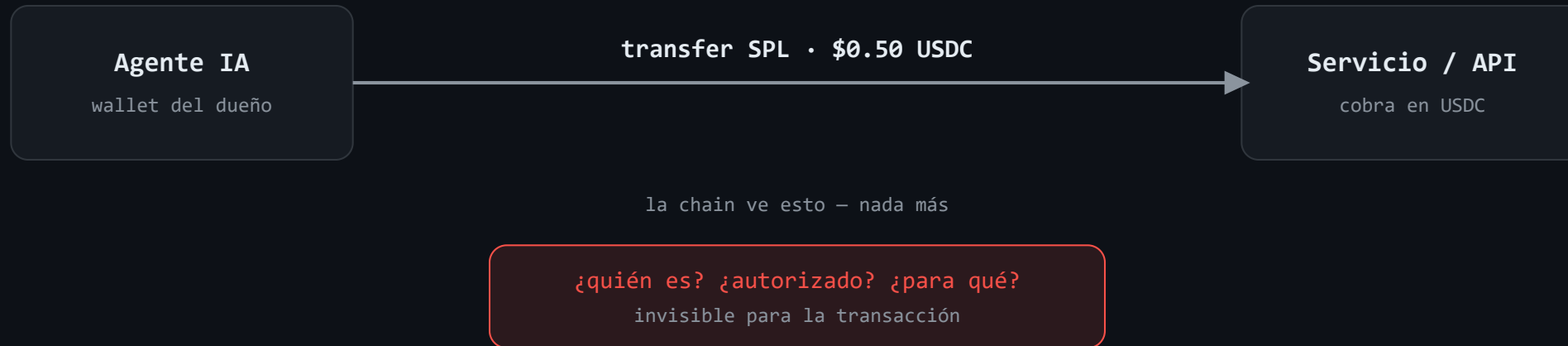
¿Qué pasó?

recibo

Una tx mueve fondos pero no dice bajo qué mandato ni para qué servicio. Sin evidencia, no hay disputa posible.

EL HUECO, VISUALMENTE

Un pago hoy: plata sí, contexto no



El ledger registra **que** la plata se movió. No registra **quién** la autorizó ni **bajo qué condiciones**. El hueco es de contexto, no de dinero.

POR QUÉ AHORA

Los rieles ganaron. Falta la cerradura.

MCPay (1ro Stablecoins) y **Latinum** (1ro AI) ya ganaron Colosseum construyendo los rails de pago x402 agente→servicio.

ERC-8004 "Trustless Agents" deployó mainnet 29/01/2026 — identidad de agente **seudónima**: su spec omite el binding a humano.

Skyfire hace KYA cerrado: JWTs en su servidor privado, suscripción, no consumible por programas, muere con la empresa.

La tubería existe y está premiada. Quién puede pasar por ella — bajo qué autorización — nadie lo resolvió on-chain.

DÓNDE ENCAJAMOS

El stack del agente que paga

Aplicaciones / merchants / APIs

lo que el agente quiere comprar

Rieles x402 – MCPay • Latinum • PayAI

la tubería que mueve la plata (ya premiada)

dnia – la cerradura: identidad + mandato + recibo

quién puede pasar por el riel, y bajo qué autorización

Solana – liquidación y evidencia pública

donde todo esto es exigible y auditable

No competimos con los rieles ni con la chain: somos la capa que les faltaba — y que cada uno puede adoptar con un hook.

LA SOLUCIÓN

Tres primitivas, un programa

① Identidad

¿quién está atrás?

Attestation **SAS** real: un issuer verificó que hay un humano detrás de la wallet. Divulgación selectiva — nivel + timestamp. **Cero PII on-chain.**

② Mandato

¿qué autorizó?

PDA firmado por el owner: máx por tx, cap diario, whitelist de payees, expiración, revocable. Los contadores los muta solo el programa.

③ Recibo

¿qué pasó?

`PaymentReceipt` emitido en la misma tx: pagador, payee, monto, mint, invoice, mandato, timestamp. Auditable para siempre.

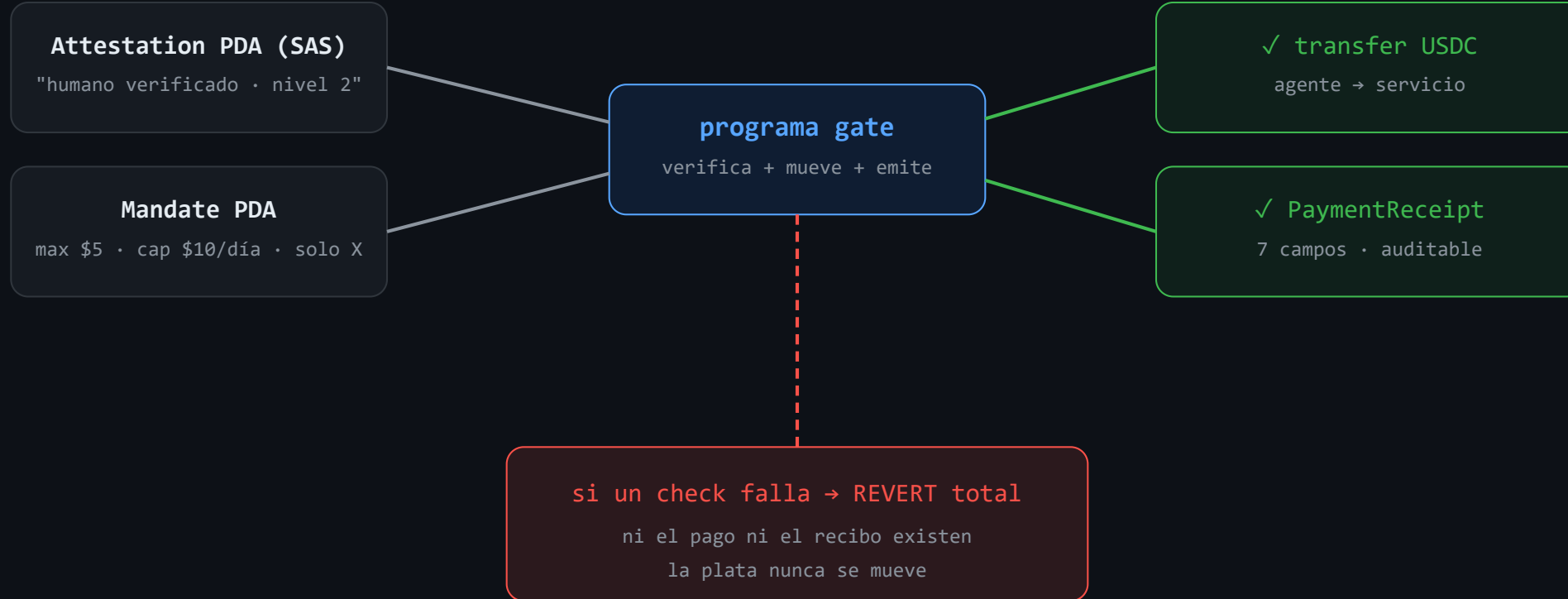
CÓMO FUNCIONA

El gate es atómico — o pasa todo, o nada

```
pay(agente, monto, servicio):  
  1. leer attestation SAS del agente → falta/vencida = REVERT  
  2. leer mandato (límite, whitelist, → violado = REVERT  
    expiry, mint = USDC esperado)  
  3. transfer USDC agente → servicio  
  4. emitir PaymentReceipt
```

Un JWT puede salteárselo un middleware. Un programa Solana **revierte la transacción misma** — **la plata nunca se mueve** si los checks fallan. No hay nada que devolver ni disputar.

Una tx: dos lecturas, un movimiento, una evidencia



6 beats, todos con signature real en devnet

Beat	Resultado on-chain
1 · Attest agente A (SAS real)	✓ attestation PDA creada
2 · Owner firma mandato (\$5/tx, \$10/día, solo X)	✓ mandate PDA
3 · Pago \$0.50 → servicio X	✓ transfer + receipt → HTTP 200
4 · Agente B sin attestation	✗ tx fallida · AttestationMissing
5 · \$10 over-limit · payee no whitelisted	✗ OverPerTxLimit · PayeeNotWhitelisted
6 · Owner revoca → siguiente pay	✗ MandateRevoked

Los rechazos aterrizan como transacciones fallidas reales — clickeables en el explorer con su `Error Code`. Y el demo entero corre con **un botón** en </demo>.

DEMO EN VIVO

Un click → 6 beats → explorer

El sitio corre la demo completa contra devnet desde el botón ► **Correr demo: terminal en vivo, signatures linkeadas, dashboard auditando en tiempo real.**

Dashboard: ledger de pagos aceptados/rechazados en criollo, estado del mandato y attestations — todo leído de la chain, nada mockeado.

Sin cuenta. Sin API key. Sin permiso.

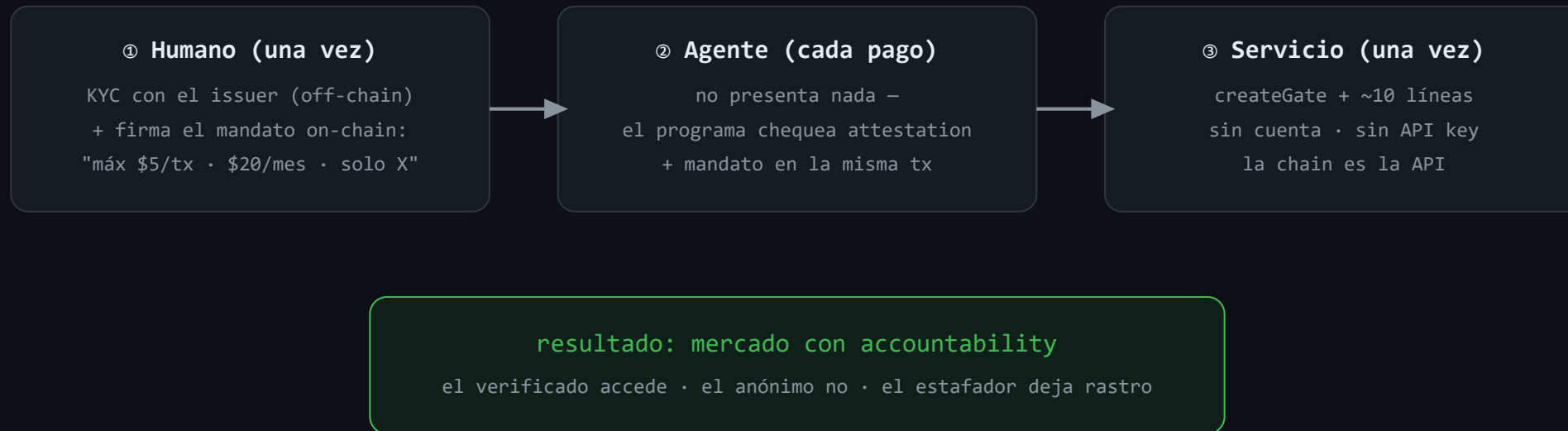
```
const gate = createGate({ programId, payee: MI_WALLET, price: 0.5, mint: USDC });

app.get("/api/premium", async (req, res) => {
  const check = await gate.verify(req.headers["x-payment"]);
  if (!check.ok) return res.status(402).json(gate.requirements());
  res.json({ resource: "...", receipt: check.receipt });
});
```

El programa, el schema SAS y el mint ya están desplegados y públicos — **la chain es la API**. El 402 es x402-shaped: encaja en el flujo de MCPay/PayAI. `demo_adoption.sh` suma un servicio nuevo en vivo.

LA VUELTA COMPLETA

Tres actores, una interacción cada uno



Accountability judicial — no vigilancia

On-chain se ve

"hay un humano verificado nivel N detrás de este agente, emitido por issuer X" — nivel, timestamp, estado de revocación. **Nada más.**

On-chain NO está

quién es el humano. La identidad real la guarda el issuer off-chain; revelarla exige **proceso legal hacia el issuer**. No hay DNI público ni trazabilidad masiva.

La credencial no te identifica — **te habilita**: el agente verificado accede a mercados que el anónimo no puede. Y si el agente estafa, el rastro del pago es la feature: el merchant lo adopta porque el recibo auditable es su defensa en una disputa.

Tres capas, tres visibilidades

PÚBLICO – la chain

"humano verificado · nivel · issuer · timestamp · revocado?"
+ recibos de pago (pagador, payee, monto, mandato)

ISSUER – off-chain

identidad real (KYC) – nunca toca la chain
distribuido por issuer, sin verificador único

REVELADO – solo por proceso legal

un juez → ordena al issuer → se sabe quién era
nadie puede reconstruir identidades desde la chain sola

Skyfire junta las dos primeras capas en un servidor privado: una empresa lo ve todo. Acá están separadas por diseño.

POSICIÓN

El ecosistema se resguarda solo

Un merchant exige humano verificado porque no quiere ser estafado — no porque una ley se lo mande.

Filipinas, Brasil y EE.UU. ya redactan esto: evidencia de que el problema es real, no la razón de existir. Si la ley nunca llega, funciona igual. Si llega — mejor una capa open-source corriendo que un registro estatal cerrado.

COMPETENCIA

**~25 lo intentaron en Colosseum. Cero ganó.
Ninguno hizo esto.**

	Skyfire KYA	ERC-8004	dnia
Humano verificado	sí	no	sí
Verificable sin cuenta	no	sí	sí
Exigible dentro del programa	no	no	sí
Sobrevive al emisor	no	sí	sí

Complementamos a 8004 (somos su human layer faltante) y a los facilitators x402 (somos un hook, no un rewrite).

MODELO — HONESTO

Protocolo abierto hoy; negocio en las orillas

La capa es pública y sin fee — la adopción vale más que el rent seeking en un estándar naciente.

Ingreso donde ya existe: verificación KYC por issuers (Persona/Veriff-class cobra por check) + integraciones premium de facilitators.

Destino probable: estándar absorbido por los rails — MCPay/Latinum ya premian la tubería; la cerradura es la pieza que falta.

ESTADO REAL

Hecho, no prometido

✓ Deployado y corriendo

Programa `D8pcktez...` + credential + schema + mint USDC-test en devnet. 21/21 tests. Demo reproducible de un click.

⚠ Declarado como mock/pendiente

Issuer KYC = mock (la capa se demuestra, no el KYC). Gate opt-in por servicio. Recibo = evento. Roadmap: issuer real, receipt PDA, bridge 8004.

DNIA

Los rails de pagos para agentes ya ganaron premios.
Nadie construyó quién puede pasar por ellos.

Nosotros construimos la cerradura.

github.com/adriangmrraa/dnia · Apache-2.0 · corré la demo en localhost:3404/demo o verificá
las signatures en el explorer.